A Hybrid of Bees Algorithm and Genetic Algorithm for the Traveling Salesman Problem

Ławrzyniak Maciej
Wrocław University of Science and Technology
Email: 284254@student.pwr.edu.pl

Jarzabek Dominika
Wrocław University of Science and Technology
Email: 284327@student.pwr.edu.pl

***Abstract*—The Traveling Salesman Problem (TSP) is a classic combinatorial optimization problem. Two popular algorithms that attempt to solve it are the Bees Algorithm (BA), which exhibits strong local exploitation capabilities around elite solutions but suffers from weak global exploration and a tendency toward premature convergence, and the Genetic Algorithm (GA), whose evolutionary operators excel at exchanging information between distant local optima and escaping stagnation. In this work, we introduce a hybrid algorithm that embeds TSP-specific crossover and mutation operators into the standard Bees Algorithm. The algorithm was tested on standard TSPLIB benchmark instances. The results demonstrate that the proposed hybrid finds shorter route lengths with faster convergence compared to standalone BA or GA implementations. This study highlights the potential of combining swarm intelligence with evolutionary operators for discrete routing problems.**

1. Introduction

The Traveling Salesman Problem (TSP) is a well-known combinatorial problem classified as NP-Complete. It requires finding a path that visits N cities exactly once and returns to the starting point, minimizing total distance traveled. Due to NP-Completeness, brute-force enumeration of all candidate routes becomes computationally infeasible as the number of nodes grows: the size of the search space is equal to $(N-1)!/2$ [1], making exhaustive search impractical even for instances as small as 20 cities.

Many metaheuristic algorithms have been developed to find near-optimal solutions to the TSP within reasonable time. Among the most prominent are evolutionary algorithms and swarm intelligence methods. The Genetic Algorithm (GA), introduced by Holland [2], operates on a population of candidate solutions by iteratively applying selection, crossover, and mutation. Crossover exchanges segments between two parent solutions to produce offspring that inherit characteristics of both, while mutation introduces random perturbations to maintain diversity. These operators give GA a strong ability to explore distant regions of the search space and escape local optima, but convergence can be slow when population diversity is exhausted.

Swarm intelligence algorithms draw inspiration from collective behaviors observed in nature. The Artificial Bee Colony (ABC) algorithm, proposed by Karaboga [3], models honey bee foraging through three types of agents: employed bees that exploit known food sources, onlooker bees that select sources based on shared quality information, and scout bees that explore randomly when a source is abandoned. While ABC has proven effective on both continuous and combinatorial problems, its canonical solution update equation was designed for continuous search spaces, and adapting it to discrete permutation-based problems such as TSP requires significant restructuring of the neighborhood search mechanism [6].

The Bees Algorithm (BA), proposed by Pham et al. [4], similarly draws inspiration from bee foraging but differs from ABC in ways that make it more naturally suited to combinatorial problems. BA explicitly distinguishes between elite and non-elite sites, assigning more forager bees to higher-quality regions, and uses a direct neighborhood search with a controllable search radius rather than a continuous difference-vector update. The neighborhood search in BA is defined operationally: any discrete move operator that produces a neighboring permutation can serve as the local search step without reformulating the algorithm's core update rule. This makes BA particularly amenable to TSP, where solutions are permutations and neighborhood moves such as swap, insertion, or reversion are naturally applicable. Despite these advantages, BA shares a weakness common to most swarm intelligence methods: it can converge prematurely when elite sites dominate recruitment, reducing population diversity and causing stagnation. GA's evolutionary operators directly address this weakness. Crossover can transfer route segments between elite solutions discovered in different regions of the search space, constructing solutions neither parent could reach through local search alone. Mutation provides an additional mechanism to escape stagnation through small, targeted perturbations.

The value of hybridizing swarm intelligence with GA operators has already been demonstrated for ABC [5], [6], [7], [8]. The present work applies the same principle to BA, which we argue is a more natural host for permutation-based operators due to its operationally defined neighborhood search. We propose a hybrid algorithm (BA-GA) that

embeds discrete crossover and mutation operators into the standard BA framework and evaluate it against standalone BA and GA baselines on TSPLIB benchmark instances.

2. Related Work on Hybrid Swarm–Evolutionary Methods

Hybridizing swarm intelligence algorithms with genetic operators has been an active research direction for TSP. Kumar et al. [5] embedded a linear crossover operator into the ABC framework, producing the Crossover-based ABC (CbABC). Evaluated on four standard benchmark functions and TSP instances of dimensions 10, 20, and 30, CbABC consistently outperformed standard ABC, with the improvement growing as problem dimension increased. Crossover probability in the range $[0.1, 0.3]$ was identified as most effective.

Maheshwari and Dutta [7] extended this direction by adding both crossover (after the employed bee phase) and uniform mutation (after the scout bee phase) to ABC. Tested on TSP instances at the same dimension levels and compared against standard ABC and a mutation-only variant, their proposal yielded the shortest mean tour lengths across all configurations, demonstrating that the combination of both operators is stronger than either in isolation.

Sabet et al. [6] proposed a Hybrid Mutation-based ABC (HMABC) for TSP, combining three distinct neighborhood operators: Neighbor Swap (NS), Neighbor Shift-Change (NSC), and a distance-weighted Neighbor-Change (NC). Validated on TSPLIB benchmarks including Berlin52, Eil51, Eil76, St70, and Rat99, the hybrid mutation scheme outperformed each individual operator and produced tours competitive with Ant Colony Optimization baselines.

Önder et al. [8] demonstrated that an ABC algorithm using swap, insertion, and reversion operators achieves a tour length of 9920 km on an 81-city real-world TSP instance of Turkey, matching a PSO-based Genetic Algorithm and outperforming an ACO reference solution by 1 km. Their result confirms that swarm intelligence with discrete operators is competitive with established evolutionary and ACO approaches even on large, real-world instances.

Collectively, these results establish that augmenting swarm intelligence algorithms with genetic operators reliably improves solution quality on TSP. The present work extends this line of research to the Bees Algorithm.

3. Methodology

3.1. Standard Bees Algorithm

The base version of Bees Algorithm uses the following parameters: population size n, the number of elite sites e and selected sites s, and their local search limits (iterations) r_e and r_s, where $r_e > r_s$.

During initialization, n random routes are generated, evaluated, and sorted from shortest to longest.

Each iteration works as follows. First, the e best individuals become elite sites. Each elite runs the 2-opt local search r_e times: in each step, it picks two random points on the route and reverses the order of cities between them. If the new route is shorter, it replaces the old one. Second, the next s individuals become selected sites and perform the same 2-opt search, but only for r_s iterations. Third, the remaining $n - e - s$ individuals act as scout bees: their routes are completely randomized to explore new areas and avoid getting stuck in local optima. After all groups are updated, the population is sorted again so the best routes become the elites for the next round. Figure 1 summarizes this procedure.

1) Initialize n random tours; evaluate all; sort ascending.
2) **Repeat** until stopping criterion:
   a) For $i = 0, \ldots, e-1$: apply 2-opt r_e times; keep best neighbor.
   b) For $i = e, \ldots, e+s-1$: apply 2-opt r_s times; keep best neighbor.
   c) For $i = e+s, \ldots, n-1$: randomize route.
   d) Re-sort population by tour length.
3) **Return** population$[0]$.

Figure 1. Standard Bees Algorithm for TSP

3.2. Genetic Algorithm

The base version of Genetic Algorithm uses population size n, crossover probability p_c, and mutation probability p_m. It also keeps a separate `bestIndividual` variable that updates whenever a better route is found, ensuring the global best is never lost.

During initialization, the algorithm generates and evaluates n random routes. The shortest route is saved as the initial `bestIndividual`.

Each iteration builds a new generation as follows. First, the current `bestIndividual` is copied directly into the new generation. The remaining $n-1$ slots are filled using binary tournament selection: two individuals are picked randomly from the current population, and the one with the shorter route becomes a parent. With probability p_c, two parents undergo Ordered Crossover (OX). The OX operator copies a segment from the first parent into the exact same positions in the child, and fills the empty spots using the order of the missing nodes from the second parent. This ensures a valid route with no duplicate cities. Both children are evaluated and added to the new generation. If crossover does not happen (probability $1-p_c$), the first parent is copied over.

Once the new generation is built, each individual has a p_m chance to undergo Partial Shuffle Mutation. This operator picks two random points and shuffles the cities between them, which changes the route differently than the 2-opt. The mutated routes are evaluated again. Finally,

`bestIndividual` is updated if any route in the new generation is shorter than the current best. Figure 2 summarizes this procedure.

1) Initialize n random tours; evaluate all; store best as `bestIndividual`.
2) **Repeat** until stopping criterion:
   a) nextGen = $\{$`bestIndividual`$\}$.
   b) While $|\text{nextGen}| < n$: select p_1, p_2 by binary tournament; with prob. p_c apply OX and add both children; else copy p_1.
   c) For each individual in nextGen: with prob. p_m apply Partial Shuffle Mutation.
   d) Update `bestIndividual` if improved.
   e) population = nextGen.
3) **Return** `bestIndividual`.

Figure 2. Genetic Algorithm for TSP

3.3. Proposed Hybrid BA-GA

The proposed hybrid algorithm (BA-GA) integrates the intensive local search capabilities of the Bees Algorithm with the global exploration mechanisms of the Genetic Algorithm. Each iteration consists of two sequential phases executed on a single shared population: a swarm-based exploitation phase followed immediately by an evolutionary exploration phase. In the BA phase, the population is sorted by tour length and partitioned into three groups: elite sites, selected sites, and scout bees, which undergo neighborhood search or randomization as described in Section III-A. Throughout this phase, the algorithm tracks and updates the global best individual. Instead of concluding the iteration at this point, the newly optimized population serves as the parent pool for the GA phase, which proceeds as described in Section III-B.

By combining these mechanisms, the hybrid approach addresses the fundamental weaknesses of its base algorithms. The BA phase rapidly refines solutions through intensive 2-opt neighborhood searches, while the subsequent GA crossover recombines these locally optimized solutions to escape region boundaries that neither parent could cross through local search alone. Meanwhile, scout bee randomization and genetic mutation together inject sufficient diversity to prevent the premature convergence characteristic of standalone swarm intelligence methods. Figure 3 summarizes this procedure.

1) Initialize n random tours; evaluate all; sort ascending.
2) **Repeat** until stopping criterion:
   a) *BA phase:*
      i) For $i = 0, \ldots, e-1$: apply 2-opt r_e times; keep best neighbor; update global best.
      ii) For $i = e, \ldots, e+s-1$: apply 2-opt r_s times; keep best neighbor; update global best.
      iii) For $i = e+s, \ldots, n-1$: randomize route.
   b) *GA phase:*
      i) nextGen $\leftarrow \{$global best$\}$.
      ii) While $|\text{nextGen}| < n$: select p_1, p_2 by binary tournament; with prob. p_c apply OX and add both children; else copy p_1.
      iii) For each individual in nextGen: with prob. p_m apply Partial Shuffle Mutation.
      iv) Update global best if improved.
      v) population $\leftarrow$ nextGen; sort ascending.
3) **Return** global best.

Figure 3. Proposed Hybrid BA-GA Algorithm for TSP

4. Benchmark Instances

All experiments use instances from the TSPLIB benchmark library [9], a standard repository of TSP instances with known optimal or best-known tour lengths. Four instances were selected to cover a range of problem sizes and structural characteristics:

eil101 [10] is a 101-city instance attributed to Christofides and Eilon [11]. Cities are distributed uniformly at random in the plane, making it a structurally homogeneous benchmark suitable for comparing baseline algorithmic behavior. The known optimal tour length is 629.

bier127 [10] is a 127-city instance based on the locations of beer gardens in Augsburg, Germany. The non-uniform, geographically motivated distribution of cities introduces clustering that tests the ability of algorithms to exploit local structure. The known optimal tour length is 118 282.

a280 [10] is a 280-city instance derived from a PCB drilling problem. Drill-hole positions in such problems tend to produce a denser and more regularly distributed point set than random Euclidean instances. The known optimal tour length is 2 579.

mu1979 [12], [13] is a 1 979-city instance derived from National Imagery and Mapping Agency (NIMA) geographic data for Oman. It represents a large-scale real-world instance and is used to assess the scalability of the proposed hybrid

under a significantly larger search space. The best-known tour length is 86 891.

Together, these four instances span one order of magnitude in problem size (101 to 1 979 cities) and three qualitatively different spatial structures: random uniform, geographically clustered, and grid-like. This diversity ensures that results are not biased toward any single instance type.

5. Experimental Setup

To ensure a fair and rigorous comparison, the standalone BA, standalone GA, and the proposed BA-GA hybrid were implemented in C++ and evaluated within the same computational environment. All algorithms were compiled in Visual Studio 2022's "Release mode" to enable standard compiler optimizations and were executed sequentially on a single thread of an Intel Core i5-12450HX processor. Given the random nature of these algorithms, each approach was executed 30 independent times per benchmark instance using distinct random seeds. The performance metrics presented in Section 6 reflects the mean best tour lengths and average computational times calculated over these 30 independent runs.

The population size was kept constant at $n = 100$ individuals across all problem instances. To comprehensively evaluate both the short-term convergence speed and the long-term stagnation avoidance of the algorithms, the experiments were conducted under two different stopping criteria: a limit of 1,000 iterations and an extended limit of 5,000 iterations. The results for these two operational scenarios are reported in separate tables in the subsequent section.

Specific algorithmic parameters were fine-tuned via an empirical grid search. For the standalone Bees Algorithm and the BA phase of the hybrid, the swarm division was fixed at $e = 10$ elite sites and $s = 30$ selected sites. To accommodate the vast differences in search space dimensions, the local search intensities were scaled accordingly. For the three smaller benchmark instances (eil101, bier127, and a280), the number of 2-opt search iterations was set to $r_e = 50$ for elite sites and $r_s = 10$ for selected sites. Conversely, for the large-scale `mu1979` instance, these limits were significantly increased to $r_e = 500$ and $r_s = 100$ to guarantee adequate exploration of the massive 2-opt neighborhood.

For the standalone Genetic Algorithm, the crossover and mutation probabilities were set to $p_c = 0.70$ and $p_m = 0.20$, respectively. However, parameter tuning for the proposed BA-GA hybrid revealed that genetic operators must be heavily constrained when applied to solutions already optimized by swarm intelligence. To prevent the destruction of highly refined edges generated during the BA phase, both evolutionary probabilities were substantially reduced. The crossover probability was decreased to $p_c = 0.25$ and the mutation rate was lowered to $p_m = 0.05$. This configuration ensures that the algorithm preserves the geometric integrity of the elite routes while still permitting occasional exchanges of high-quality segments.

6. Results

The results are divided into two parts based on the iteration limit: a short-term execution limited to 1,000 iterations (Table 1) and an extended execution limited to 5,000 iterations (Table 2). Each table reports the mean best tour length and the average computational time (in seconds) evaluated over 30 independent runs for each of the algorithms. For clarity of presentation, the average execution times are rounded to two decimal places, while the mean tour lengths are rounded to the nearest integer.

TABLE 1. EXPERIMENT RESULTS FOR 1,000 ITERATIONS (MEAN OF 30 RUNS)

		GA		**BA**		**BA-GA**	
Instance	Optimal	Tour	Time	Tour	Time	Tour	Time
eil101	629	1484	0.06	680	0.24	695	0.32
bier127	118282	304285	0.08	126339	0.28	128276	0.34
a280	2579	17162	0.13	3951	0.52	2957	0.63
mu1979	86891	4069520	0.88	915149	41.80	663882	41.61

TABLE 2. EXPERIMENT RESULTS FOR 5,000 ITERATIONS (MEAN OF 30 RUNS)

		GA		**BA**		**BA-GA**	
Instance	Optimal	Tour	Time	Tour	Time	Tour	Time
eil101	629	1303	0.35	675	1.15	691	1.39
bier127	118282	268926	0.39	123722	1.39	127980	1.64
a280	2579	13758	0.70	2898	2.60	2945	3.09
mu1979	86891	3483180	5.39	154767	193.84	102344	189.46

7. Discussion

The results reveal a consistent and interpretable pattern across all four benchmark instances: the relative advantage of the BA-GA hybrid over the standalone BA grows with problem size, while on smaller instances the hybrid offers no improvement and may even perform marginally worse.

The standalone GA performs poorly across all instances and both iteration limits, producing tours that are often an order of magnitude longer than the optimum. This confirms that pure evolutionary search without a local refinement mechanism is insufficient for TSP at the population sizes and iteration counts used here. GA's inability to construct competitive solutions stems from the absence of any structure-preserving local search: crossover and mutation alone cannot efficiently navigate a permutation space of this complexity. Its results therefore serve primarily as a lower-bound reference that contextualizes the contribution of the BA component.

On the two smallest instances, eil101 and bier127, the standalone BA consistently outperforms the hybrid at both iteration limits. On eil101 at 5 000 iterations, BA achieves a mean tour of 675 against BA-GA's 691, and on bier127 the gap widens slightly from 1 000 to 5 000 iterations (123 722

versus 127 980). This result suggests that on instances of this scale, the 2-opt neighborhood search is sufficient to refine solutions to a quality that the OX crossover subsequently disrupts rather than improves. The crossover recombines two locally optimized permutations, but for short tours the segments inherited from the second parent are likely to break edge sequences that 2-opt has already carefully assembled. The reduced crossover probability $p_c = 0.25$ mitigates but does not fully eliminate this effect at small scales.

The a280 instance presents a transitional behavior. At 1 000 iterations, BA-GA achieves a mean tour of 2 957 against BA's 3 951, a 25% improvement, indicating that crossover provides a meaningful advantage in early convergence. At 5 000 iterations, however, BA narrows the gap to near parity (2 898 versus 2 945), suggesting that the standalone BA can eventually reach comparable quality given sufficient time. The hybrid therefore offers a convergence speed advantage on medium-scale instances that diminishes as the iteration budget grows.

The most pronounced and consistent advantage of the hybrid emerges on the large-scale mu1979 instance. At 1 000 iterations, BA-GA produces a mean tour of 663 882 against BA's 915 149, a 27% improvement. At 5 000 iterations the gap increases further: BA-GA achieves 102 344 against BA's 154 767, a 34% improvement, representing a reduction from 78% to 18% above the known optimum. This scaling behavior is consistent with the theoretical motivation for the hybrid: on a large search space, the BA phase alone becomes trapped in geographically isolated local optima, and the crossover operator provides the only mechanism capable of transferring high-quality route segments across distant regions of the population. Notably, BA-GA also runs slightly faster than standalone BA on mu1979 at both iteration limits (41.61 s versus 41.80 s at 1 000; 189.46 s versus 193.84 s at 5 000), indicating that the GA phase does not add meaningful computational overhead and partially offsets the cost of the most expensive neighborhood searches.

8. Conclusions

This paper proposed BA-GA, a hybrid algorithm that embeds Ordered Crossover and Partial Shuffle Mutation from the Genetic Algorithm into the Bees Algorithm framework for the Traveling Salesman Problem. Experiments on four TSPLIB instances of increasing scale, evaluated over 30 independent runs at two iteration limits, produced the following findings.

First, the hybrid consistently and significantly outperforms both standalone algorithms on the large-scale mu1979 instance, reducing the mean tour length by up to 34% relative to the standalone BA and achieving results 18% above the known optimum at 5 000 iterations. This confirms the primary hypothesis that crossover-driven information exchange between locally refined solutions provides a meaningful escape from stagnation when the search space is large.

Second, the advantage of the hybrid is instance-scale dependent. On small instances such as eil101 and bier127, the standalone BA slightly outperforms BA-GA, indicating that the OX crossover disrupts edge sequences that 2-opt has already refined. On the medium-scale a280 instance, BA-GA converges faster but the standalone BA reaches comparable quality given sufficient iterations. These results suggest that the hybrid is most beneficial when the local search alone cannot adequately cover the neighborhood of elite solutions within a practical iteration budget.

Third, the hybrid introduces negligible computational overhead compared to the standalone BA, and on the largest instance is marginally faster, confirming that the GA phase is not a bottleneck.

The scale-dependent nature of the results points to several directions for future work. An adaptive crossover probability that decreases as the population converges, or that scales with problem size, may preserve the benefits of crossover on large instances while avoiding disruption on small ones. Additionally, replacing the uniform Partial Shuffle Mutation with a distance-aware operator such as the Neighbor-Change proposed by Sabet et al. [6] may improve solution quality across all scales.

9. Contributions

Maciej Ławrzyniak was primarily responsible for writing the paper and performing code reviews and corrections of the C++ implementation. Dominika Jarzabek was primarily responsible for developing the C++ codebase and performing text corrections and proofreading of the article. The complete source code developed for this study is publicly available at https://github.com/Othalax/TSPOptimizationComparison.

References

[1] A. Ugur and D. Aydin, "An interactive simulation and analysis software for solving TSP using ant colony optimization algorithm," *Advances in Engineering Software*, vol. 40, pp. 341–349, 2009.

[2] J. H. Holland, *Adaptation in Natural and Artificial Systems*. Ann Arbor: University of Michigan Press, 1975.

[3] D. Karaboga, "An idea based on honey bee swarm for numerical optimization," Erciyes University, Engineering Faculty, Technical Report TR06, 2005.

[4] D. T. Pham, A. Ghanbarzadeh, E. Koç, S. Otri, S. Rani, and M. Zaidi, "The Bees Algorithm – a novel tool for complex optimisation problems," in *Proc. IPROMS 2006 Innovative Production Machines and Systems Virtual Conference*, pp. 454–461, 2006.

[5] S. Kumar, V. K. Sharma, and R. Kumari, "A novel hybrid crossover based artificial bee colony algorithm for optimization problem," *International Journal of Computer Applications*, vol. 82, no. 8, pp. 18–25, Nov. 2013.

[6] S. Sabet, F. Farokhi, and M. Shokouhifar, "A hybrid mutation-based artificial bee colony for traveling salesman problem," *Lecture Notes on Information Theory*, vol. 1, no. 3, pp. 99–103, Sep. 2013.

[7] V. Maheshwari and U. Dutta, "Enhanced artificial bee colony algorithm for travelling salesman problem using crossover and mutation," *International Journal of Computer Applications*, vol. 91, no. 13, pp. 37–40, Apr. 2014.

[8] E. Önder, M. Özdemir, and B. F. Yıldırım, "Combinatorial optimization using artificial bee colony algorithm and particle swarm optimization supported genetic algorithm," *Kafkas University Journal of Economics and Administrative Sciences Faculty*, vol. 4, no. 6, pp. 59–70, 2013.

[9] G. Reinelt, "TSPLIB — a traveling salesman problem library," *ORSA Journal on Computing*, vol. 3, no. 4, 1991.

[10] F. Greco, Ed., *Travelling Salesman Problem*. Vienna, Austria: In-Tech Education and Publishing, 2008, ISBN 978-953-7619-10-7.

[11] N. Christofides and S. Eilon, "An algorithm for the vehicle-dispatching problem," *Operational Research Quarterly*, vol. 20, no. 3, pp. 309–318, 1969.

[12] D. L. Applegate, R. E. Bixby, V. Chvátal, and W. J. Cook, *The Traveling Salesman Problem: A Computational Study*. Princeton, NJ: Princeton University Press, 2006.

[13] W. J. Cook et al., "National TSPs," *The Traveling Salesman Problem Home Page*. [Online]. Available: https://www.math.uwaterloo.ca/tsp/world/countries.html.